

S

COMPLETE PRODUCT & KNOWLEDGE GUIDE · V2.0 · SEPTEMBER 2026

Take back control of every **subscription** you pay for.

Subscrr is a subscription intelligence platform: it shows individuals and teams what they **really** spend on recurring services — in one dashboard, in any currency — and warns them **before** the next charge happens. This edition covers every feature in depth: how each screen works, what every metric means, the notification and email systems, the AI scanner, the Payload CMS content platform, the security model, deployment, and answers to common questions.

19 pages · full feature walkthrough with 18 real product screenshots ·

metric glossaries · daily workflows · CMS administration · security & privacy · deployment · FAQ

CONTENTS

What's inside

01 · What Subscrr does	The problem, the four pillars, the value
02 · Getting started	Sign-up to first subscription in five steps
03 · The dashboard	Every metric defined — the numbers glossary
04 · Managing subscriptions	CRUD, filters, pause, export — the full workflow
05 · AI Receipt Scanner	What the AI reads, what it never does
06 · Analytics & optimization	Projections, categories, cost suggestions
07 · Renewal calendar	Every charge mapped to its day
08 · Notifications & email	Alert types, timing, delivery channels
09 · Settings & multi-currency	Currencies, budgets, data portability
10 · Admin portal	Platform health at a glance
11 · Payload CMS — overview	Where it is, how to log in, what it powers
12 · Payload CMS — daily workflows	Publishing, media, drafts, versions
13 · Security & privacy	Every protection, explained
14 · How Subscrr compares	vs bank aggregation and manual ledgers
15 · Under the hood	Architecture, data model, deployment
16 · FAQ & troubleshooting	The questions everyone asks

Reading paths: *Evaluators* → sections 01–08, 14 · *Operators/content teams* → 09–12 · *Technical reviewers* → 13, 15, 16. Every screenshot in this guide was captured from the running product — nothing is mocked.

What Subscrr does

Banks show you money **after** it leaves. Subscrr shows you **before** — that one-day-early warning is the entire value proposition.

1

Track everything recurring

Every subscription in one ledger: price, currency, billing cycle, category, payment method, renewal date, notes. Add manually in seconds, paste a confirmation email for parsing, or drop a receipt image into the AI Scanner.

2

See the honest numbers

Yearly plans are normalised to true monthly equivalents, mixed currencies convert to one display currency using live exchange rates, and the real 12-month commitment is always visible — not hidden in twelve separate charges.

3

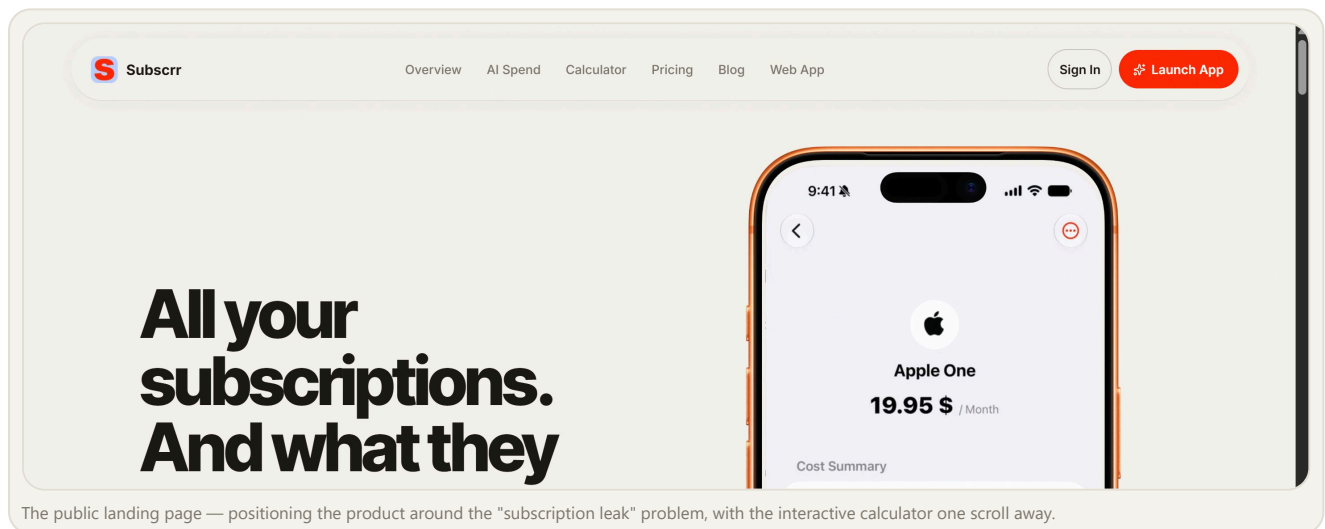
Never be surprised

A background engine runs daily and generates renewal alerts (3 days and 1 day before), free-trial expiry warnings, and budget-threshold alerts — delivered as in-app notifications *and* real emails.

4

Analyse & optimise

Category breakdowns, a 12-month spend projection, top-expense ranking, payment-method mix, and concrete cost-optimization suggestions quantify exactly which subscriptions to cancel or switch to annual billing.



The public landing page — positioning the product around the "subscription leak" problem, with the interactive calculator one scroll away.

The problem in numbers: industry studies consistently put forgotten-subscription waste at \$200+ per person per year — mostly from trials that converted silently and annual renewals nobody remembered. Subscrr's alert engine exists to make that category of loss impossible.

Getting started — five minutes to first value

1

Create your account

`/register` — email, password (8+ characters), display name. Passwords are bcrypt-hashed; login is rate-limited and protected by per-account lockout. Forgot your password later? The reset flow emails a real, single-use link valid for one hour.

2

Set your currency and budget

`/dashboard/settings` — pick your display currency (USD, EUR, GBP, INR, JPY, CAD, AUD) and set a monthly budget target. Everything on the dashboard is measured against this number.

3

Add your subscriptions — three ways

Manual entry (name, amount, cycle, renewal date, category, colour) · paste billing text from any confirmation email into the Scanner's text tab · or drop a receipt/invoice image into the AI Scanner and let the vision model fill the form.

4

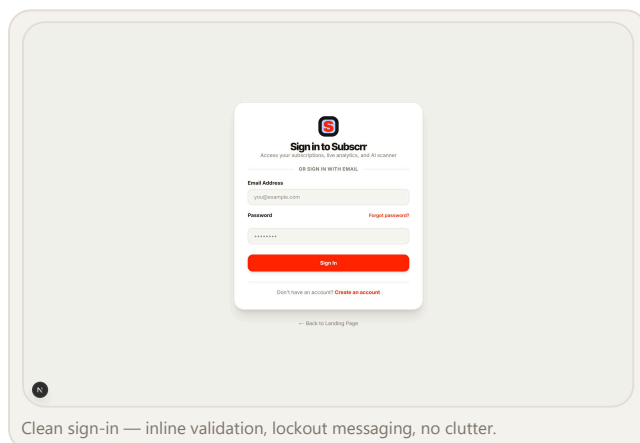
Read your dashboard

Monthly spend, annual commitment, daily burn, next charge — recalculated live. Yearly plans appear as their true monthly equivalent, so nothing hides in the annual layer.

5

Let the alerts work

From then on the daily engine watches every renewal date, trial end and your budget line. You act on a notification — or deliberately ignore one. Cancelling becomes a decision, never a discovery.

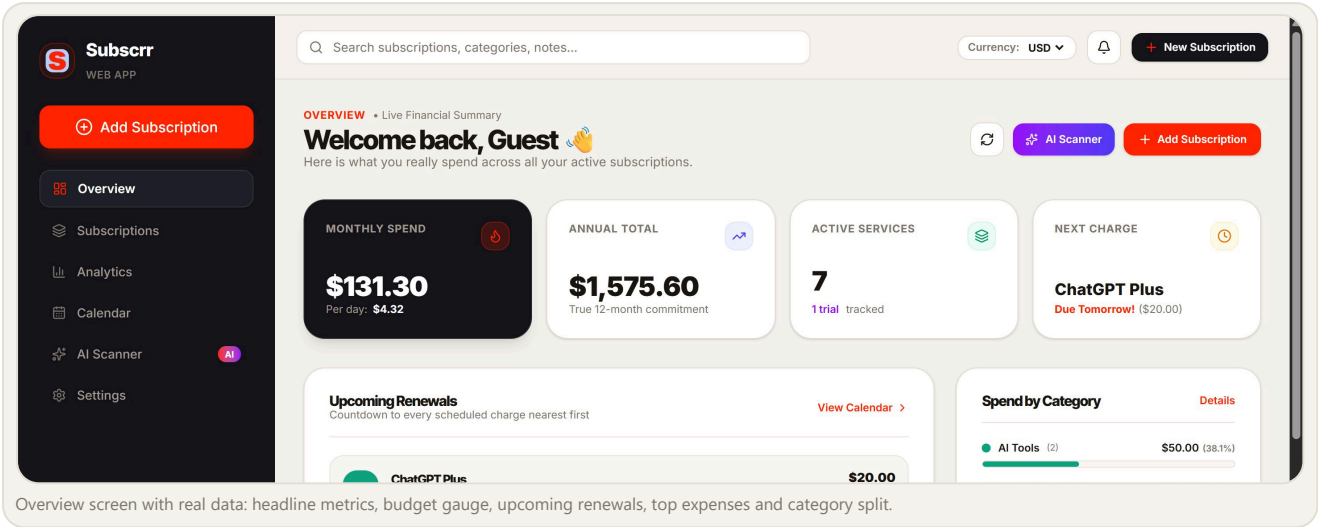


Clean sign-in — inline validation, lockout messaging, no clutter.

What you never have to do

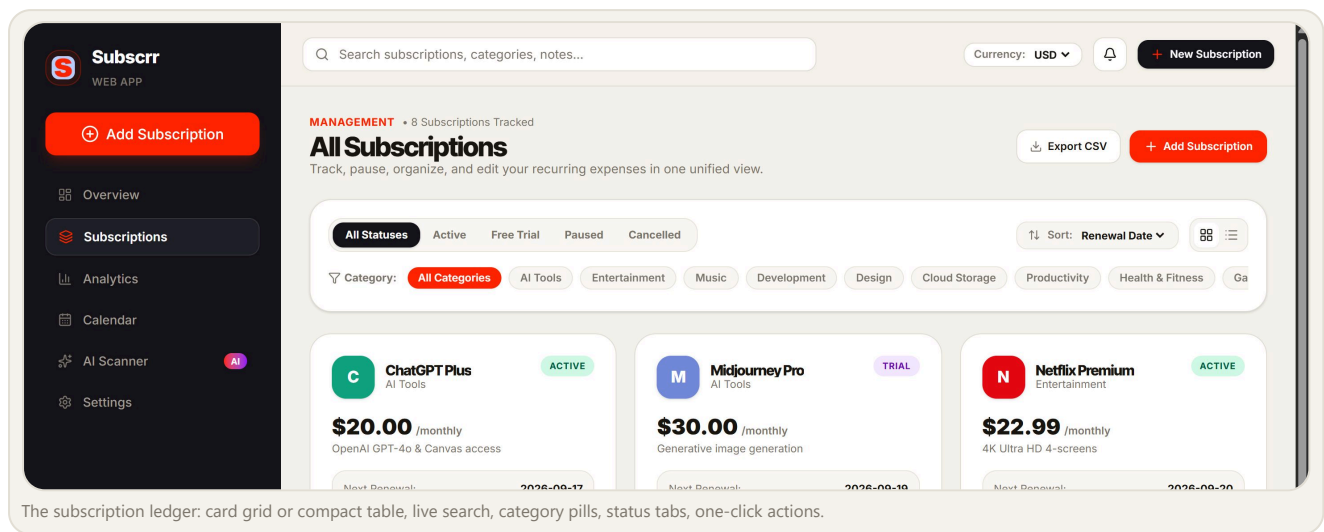
Connect a bank account. Install anything. Trust a third-party aggregator with read access to your finances. Subscr works on what *you* tell it — and extracts it for you from the documents you already have.

The dashboard — every metric, defined



METRIC	EXACTLY WHAT IT MEANS
Monthly Spend	All active subscriptions normalised to a per-month figure. A \$100/year plan contributes \$8.33; a \$10/week plan contributes \$43.33. This is the one number that answers "what do subscriptions cost me per month?" honestly.
Annual Commitment	Monthly Spend × 12 — the true 12-month run-rate. The number that changes behaviour: \$60/month feels different from \$720/year.
Daily Burn	Annual Commitment ÷ 365. The psychology hook: framed as cash leaving your wallet every single morning, small subscriptions become visible.
Active Services	Count of subscriptions by state: active, paused (excluded from totals), trial (flagged for expiry alerts), cancelled (kept for history only).
Next Charge	The nearest upcoming renewal with a live countdown — "in 3 days" turns an abstract ledger into a concrete calendar.
Budget gauge	Monthly Spend against your target, turning amber past 80% and firing a budget alert in the notification system.

Managing subscriptions — the full workflow



Find anything fast

Real-time search across names and notes; filter by category (AI Tools, Entertainment, Music, Development, Design, Cloud, Productivity, Health, Gaming, Utilities); tab by status (Active / Trial / Paused / Cancelled); sort by renewal date, amount, name or date added.

Pause, don't lose

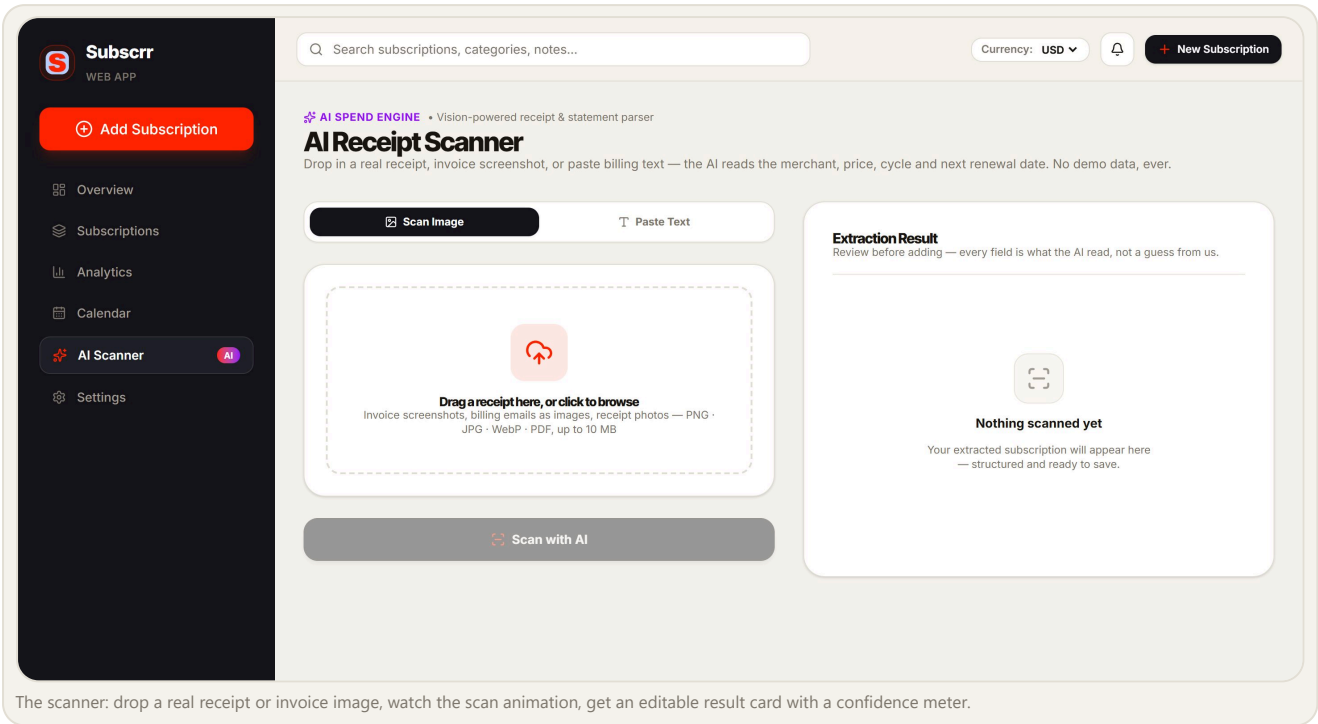
Pausing freezes a subscription's impact on every total without deleting its history — perfect for seasonal services. Resuming restores it instantly. Cancellation keeps the record for your annual review.

Own your data

One-click export of the full ledger to CSV (spreadsheet-ready, formula-injection-sanitised) or JSON (backup / import-ready). No lock-in: your financial data leaves whenever you ask it to.

The editing model: every field is editable after creation — when Netflix raises prices or you switch a plan from monthly to annual, edit the subscription and every metric, projection, calendar entry and alert recomputes immediately. Amounts are stored as integer cents, so there is no rounding drift anywhere in the system.

AI Receipt Scanner — real extraction, zero fabrication



The scanner: drop a real receipt or invoice image, watch the scan animation, get an editable result card with a confidence meter.

What the AI reads

Merchant/service name · the **actually printed amount** · currency symbol or code · billing cycle (daily → yearly) · likely category and brand colour · next renewal date · and its own confidence score, shown to you honestly.

What it never does

Invent data. Non-subscription documents produce a clear error. Amounts it cannot determine come back as zero with a "confirm this" note — never a plausible-looking guess. Demo/sample receipts do not exist in the product.

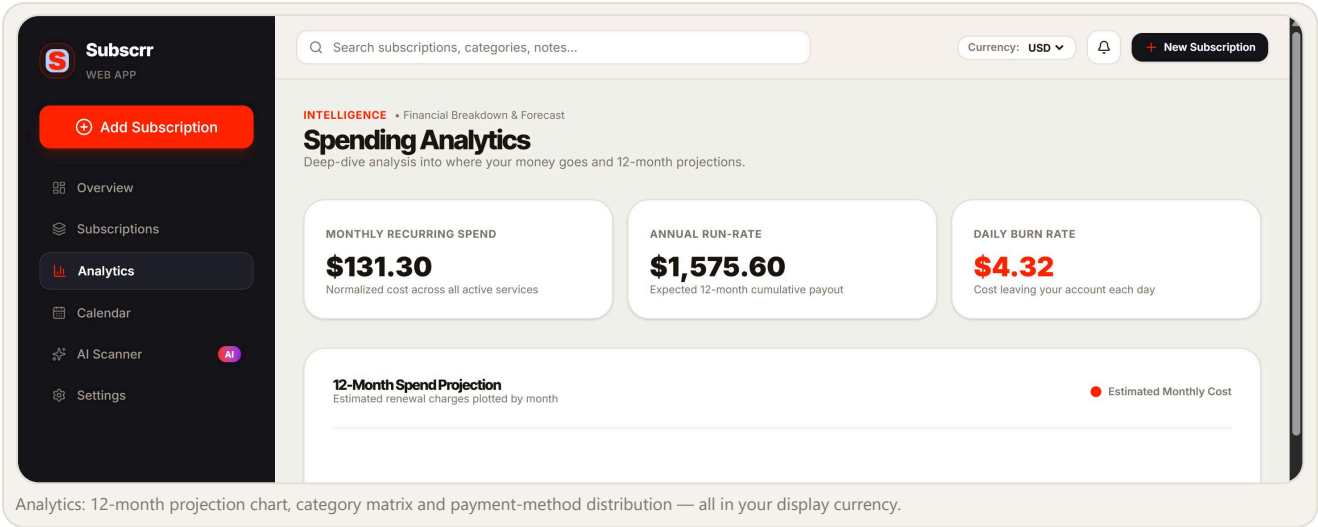
Two input modes

Image tab: drag-and-drop or browse — a scan-line animation plays while the vision model reads the document. **Text tab:** paste any confirmation email or bank statement line — parsed locally, clearly labelled "Basic local parse" in the engine badge.

One click to ledger

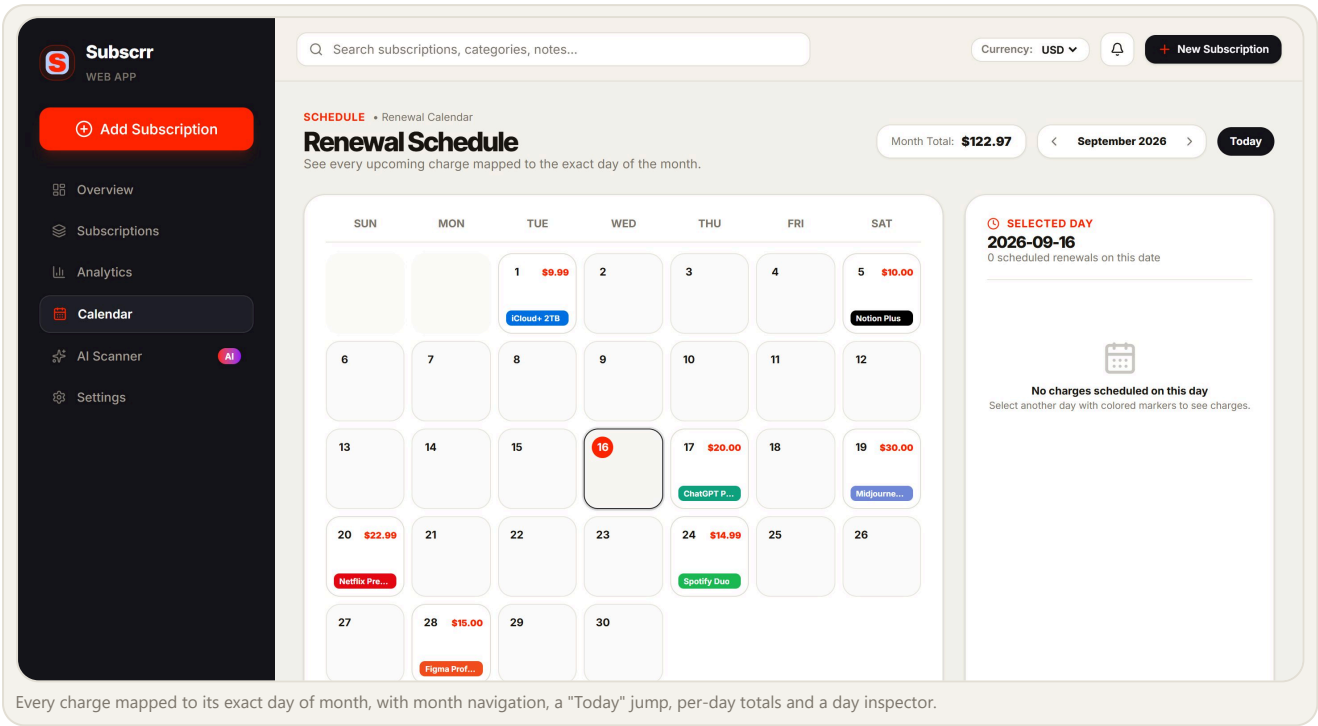
The result card is fully editable before saving. Confirm, and the subscription lands in the ledger — dashboard, analytics, calendar and alerts all update immediately. The average add takes under ten seconds.

Analytics & cost optimization



SECTION	WHAT IT TELLS YOU
12-month projection	Forward simulation of every renewal by month. Months with stacked annual renewals spike visibly — the "why is March so expensive" question answered before March.
Category matrix	Per category: monthly spend, percentage share, item count and annualised impact. Sorted by weight — Entertainment vs AI Tools vs Cloud, quantified.
Payment-method mix	Spending split by card, PayPal, Apple/Google Pay and bank transfer — useful when a card expires or you rotate payment methods.
Optimization suggestions	Concrete, computed from your data: annual-billing savings on plans you pay monthly (typically 15–20%), consolidation candidates among overlapping AI tools, trial-ending warnings, and your most expensive single services.

The renewal calendar



Read the month like a bill

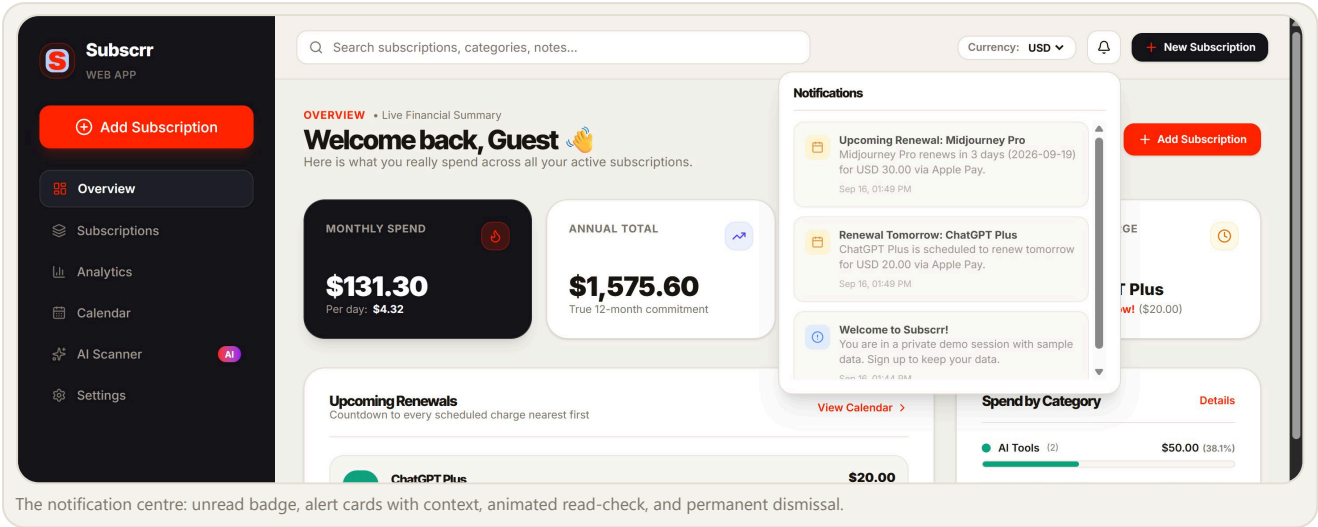
The calendar transforms the ledger into cash-flow reality: which days are heavy, which are clear, and the total commitment for the selected month at the top.

Click any day

The day inspector lists every charge due that day with amounts and payment methods, plus the day's total — the exact brief you need before a payday.

Why it matters: most subscription pain comes from *timing*, not amount — three annual renewals landing in the same week. The calendar makes those collisions visible months ahead, while you can still spread them.

Notifications & email — the alert engine



ALERT TYPE	WHEN IT FIRES
Renewal alert	3 days and again 1 day before each active subscription's renewal — the "do you even use this?" checkpoint.
Trial alert	When a free trial's end date approaches — the last honest chance to decide before conversion.
Budget alert	When monthly spend crosses 80% of your target, and again at 100%.
System / welcome	Onboarding confirmations and platform messages.

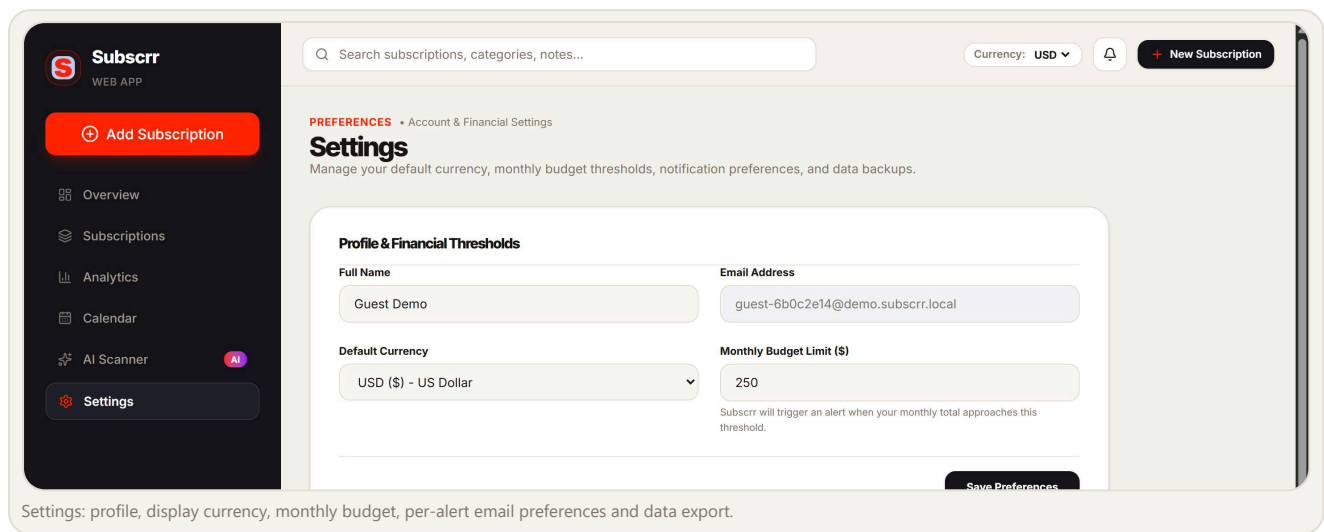
Two delivery channels

Every alert arrives **in-app** (bell dropdown with unread badge) and as a **real branded email** when a provider is configured. Each channel can be toggled per alert type in Settings — silence email, keep in-app, or vice versa.

Interaction is deliberate

Clicking a notification plays an animated check-mark and slides it out — a **permanent** dismissal, not a cosmetic fade. Read items offer an X for bulk cleanup. Nothing reappears to nag you twice.

Settings, currencies & data portability



Seven currencies, live rates

USD, EUR, GBP, INR, JPY, CAD, AUD.

Amounts are stored per-subscription in their original currency and integer cents; conversion to your display currency uses live exchange rates, with resilient fallback rates if the rate service is unreachable.

Budget that means something

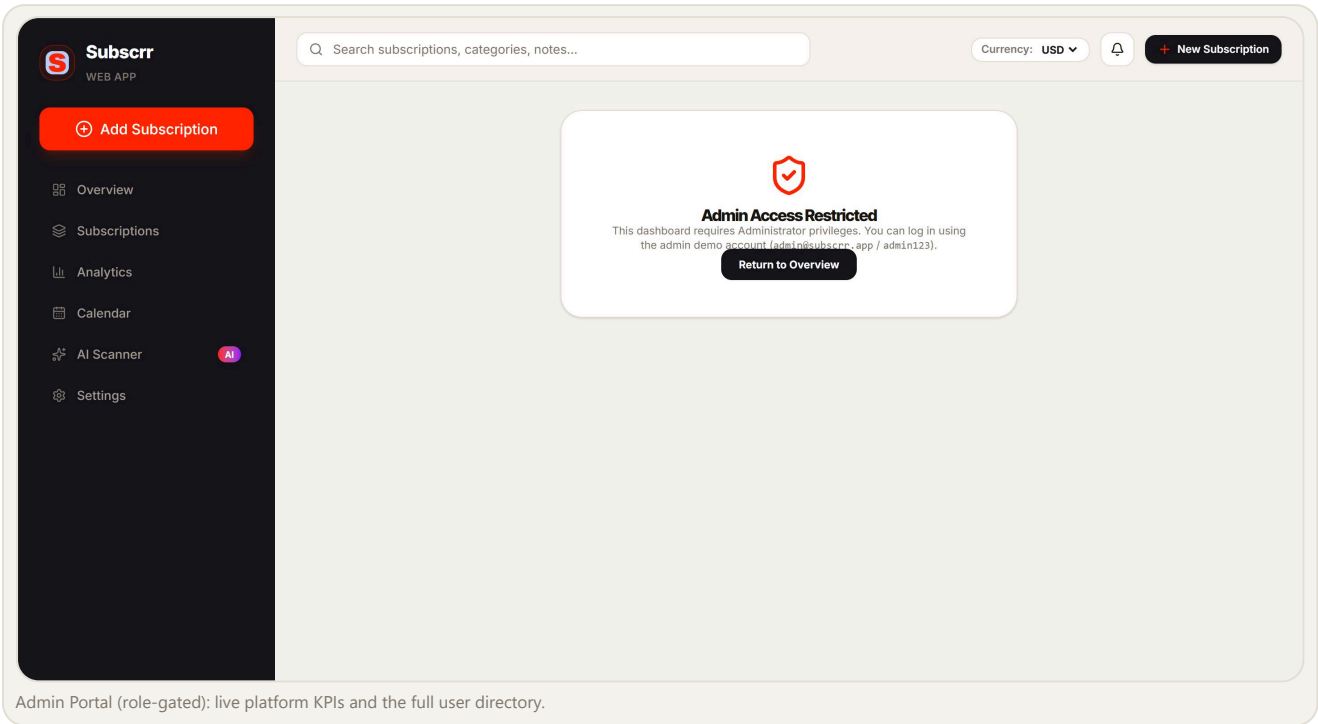
The monthly budget target feeds the dashboard gauge and the budget-alert engine — set it once, and every month the system watches the line for you.

Email preferences

Per-alert-type toggles for the email channel: renewals, trials, budget, product. Unsubscribe from email noise without losing in-app alerts.

Data portability: full CSV and JSON export of your ledger is one click away in Subscriptions and Settings. Your financial record is yours — no support ticket, no export fee, no lock-in. Combined with self-hostable SQLite storage, "what happens if I leave?" has a boring, correct answer: you take the file.

The admin portal — platform health at a glance



WHAT YOU SEE	WHY IT MATTERS
Registered users	Total accounts — the topline growth number.
Tracked subscriptions / active	Total subscriptions across all users and how many are currently active — engagement measured in data, not logins.
Platform MRC	The monthly recurring cost tracked across the entire platform — the "value under management" of the product, and the metric investors ask about.
Waitlist leads	Early-access signups captured by the landing page — the live sales funnel.
User directory	Every account with role, currency, subscription count and join date. Deliberately does <i>not</i> expose another user's subscription details — privacy is a feature, not an omission.

Payload CMS — the built-in blog engine

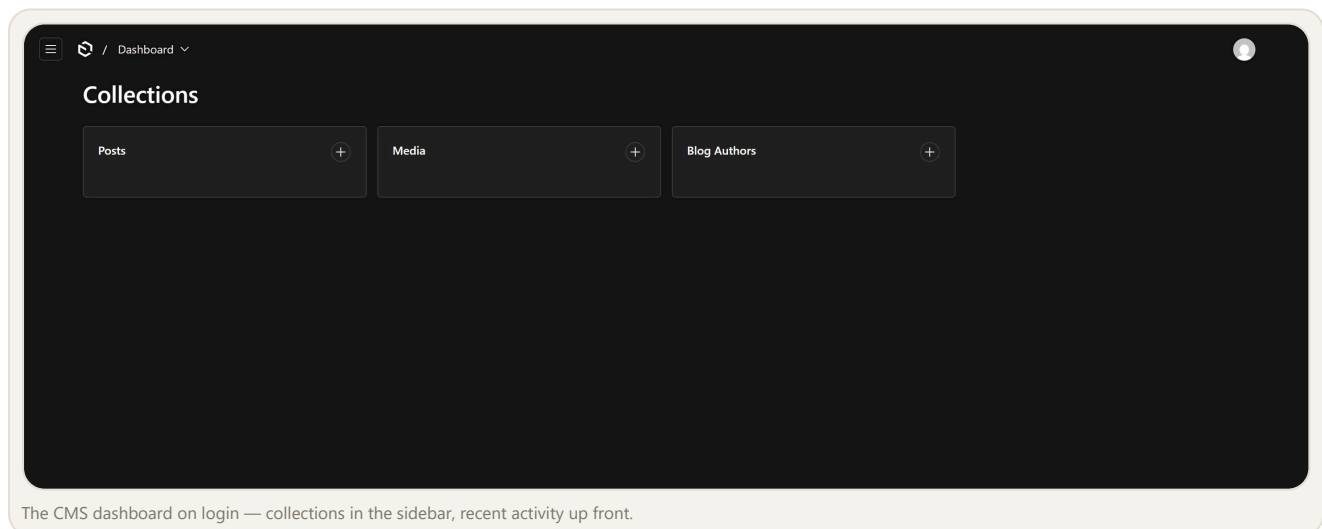
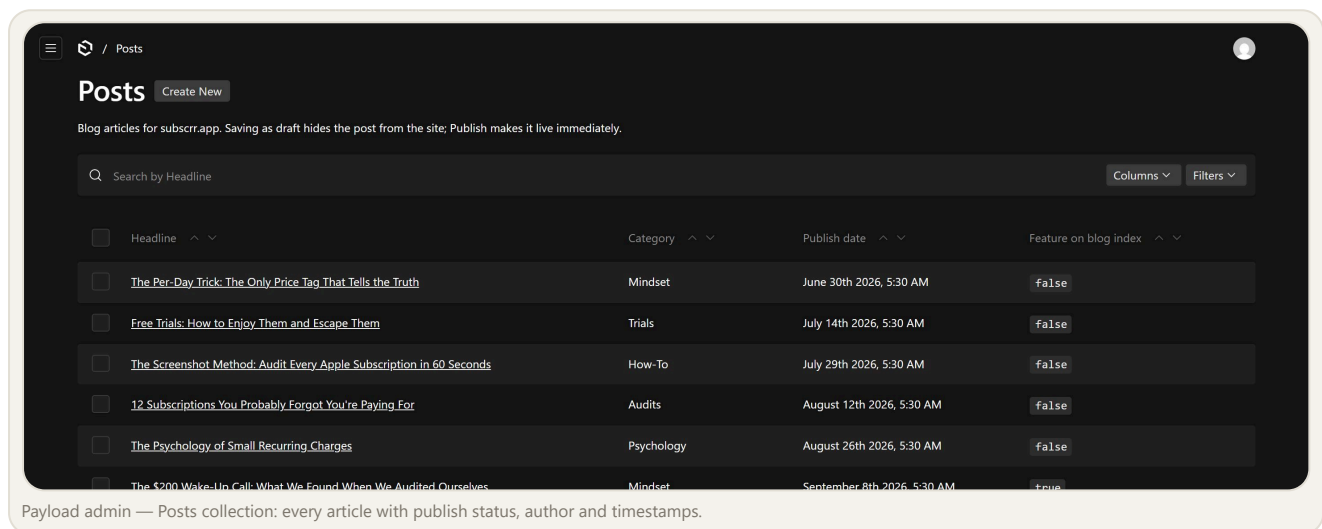
The marketing blog runs on **Payload CMS 3**, embedded directly inside the Next.js application — a database-backed, admin-editable content system. Publishing an article requires no code and no deploy.

Where it is & how to log in:

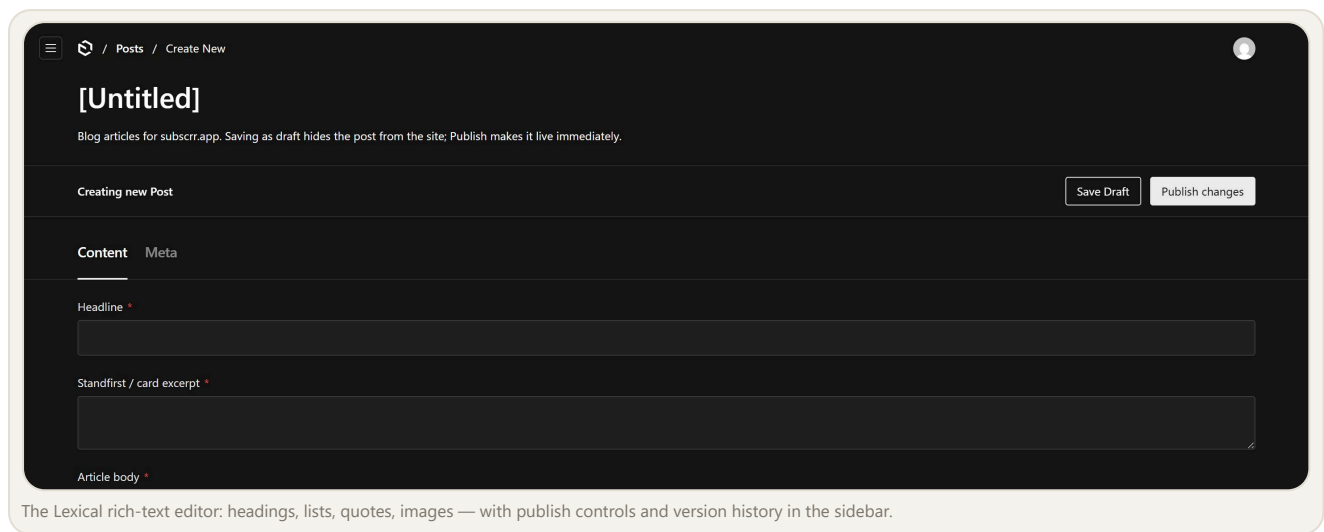
Admin panel URL: `/admin` on your deployment (e.g. `http://localhost:3000/admin`).

CMS admin account (deliberately separate from app users): `admin@subscrr.local` — password set via the `PAYLOAD_ADMIN_PASSWORD` environment variable (dev default: `ChangeMe-PayLoad-2026!`, change before any real deployment).

The six launch articles are seeded automatically on first boot; publishing edits is instant, with version history behind every document.



Payload CMS — daily publishing workflows



1

Publish a new article

Posts → Create New → title, excerpt, slug (auto), category, tags, publish date, read-minutes, accent colour → upload a hero image in the Media field → write the body in the editor →

Publish changes

. It appears on `/blog` within ~60 seconds — no rebuild.

2

Work in drafts

Save Draft keeps an article invisible to the public site while you edit. The status column shows published vs draft at a glance. Every save creates a version — restore any earlier state from the sidebar.

3

Manage media

The Media collection stores images with required alt text (accessibility by design); responsive card/hero/thumb variants are generated automatically and served through the app's own pipeline — no external image host.

4

Content is API-first

Everything is also readable over REST (`/api/payload/posts`) and GraphQL (`/graphql`) — the same content can feed newsletters, apps or social automation later.

Security & privacy — every protection, explained

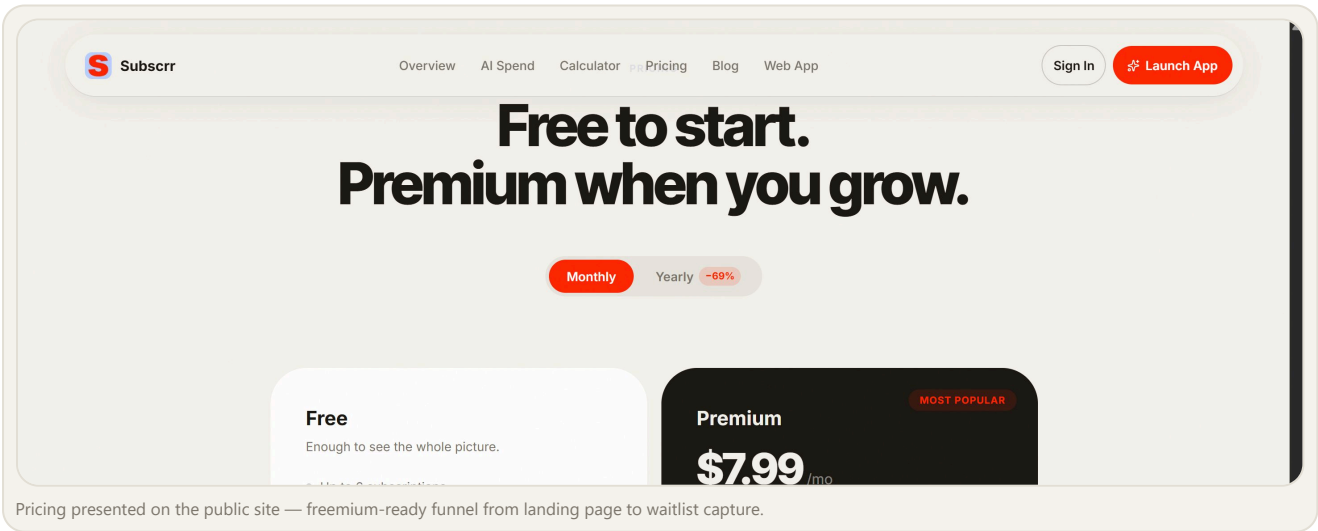
LAYER	WHAT IS ACTUALLY ENFORCED
Credentials	Passwords hashed with bcrypt — never stored or logged in plain text. JWTs are signed with a pinned HS256 algorithm and a ≥32-character secret; production refuses to boot without one.
Brute-force defence	Two independent layers: IP rate limiting (login attempts budgeted per window) and per-account lockout — five failed passwords lock the account for a cool-off period, protecting one targeted victim even from distributed attempts.
Password reset	Tokens are stored hashed (a database leak reveals nothing usable), single-use, and expire after one hour. The reset page never discloses whether an email exists.
Session validation	Every protected request carries a Bearer token verified server-side; stale sessions are cleared silently. Page-load revalidation runs on a separate rate budget so normal usage never locks you out.
Private files	Uploaded receipts are served only with a valid token — anonymous enumeration of the uploads directory is impossible.
Input & output safety	Every write validated by strict schemas; every SQL query parameterised (no injection surface); CSV exports sanitised against formula injection when opened in Excel.
Transport & headers	HSTS, strict Content-Security-Policy on API surfaces, no server fingerprinting, explicit CORS allowlist — never a wildcard in production.
Money integrity	Amounts stored as integer cents end-to-end — no floating-point drift can ever distort a total.
Data ownership	Self-hostable SQLite files, full export anytime, and no bank-credential aggregation anywhere in the product. The attack surface of "nothing to connect" is zero.

How Subscrr compares

The subscription-tracking space splits into bank-aggregation tools and manual ledger apps. Subscrr takes a third path: manual + AI-assisted tracking with a privacy-first, multi-currency core — and a complete content platform in the same deploy.

	SUBSCRR	BANK-AGGREGATION APPS (Rocket Money, Copilot)	MANUAL LEDGER APPS (Bobby, Notion trackers)
Works without bank credentials	Yes — nothing to connect	No — requires read access to accounts	Yes
AI receipt / invoice extraction	Yes — vision model, real amounts, confidence score	No — transaction matching only	No
Proactive renewal & trial alerts by email	Yes — before the charge, both channels	Partial	Rarely / manual
True multi-currency with live conversion	Yes — per-subscription, one display total	Limited to bank locale	Usually static
Built-in CMS blog for the brand	Yes — Payload, embedded, versioned	No	No
Self-hostable / data ownership	Yes — SQLite files, full export, containerised	No — cloud only	Varies
12-month cash-flow projection	Yes — calendar + chart	Limited	No
Platform admin & waitlist funnel	Yes — built in	N/A	No

The differentiator in one line: Subscrr delivers the foresight of bank-aggregation tools without asking for bank access — using AI extraction on the documents you already have — and packages the brand's content platform in the same deployment.



Pricing presented on the public site — freemium-ready funnel from landing page to waitlist capture.

Technical architecture & deployment

Stack

Next.js 16 (React 19, Tailwind 4) frontend · Express 4 + TypeScript API with Zod validation · SQLite with versioned migrations and integer-cent money · Payload 3 CMS with generated migrations · Nodemailer/Resend email · OpenAI-compatible vision · Docker + GitHub Actions CI.

One service, one port

A unified server (`server.mjs`) mounts the compiled Express API and the compiled Next.js app — including Payload's admin, REST and GraphQL — on a single HTTP listener. One deploy, one URL, one health check (`/api/health` verifies the database live).

Two data stores

App data (users, subscriptions, notifications, activity, leads) and content data (posts, media, authors) live in separate SQLite databases — a data breach or corruption in one never touches the other. Both survive redeploys on a persistent volume.

Operations

Daily alert cron at 00:05 · idempotent migrations and seeds on every boot · rate limiting scoped per surface · structured activity logging · automated CI (typecheck, 35-test integration suite, builds, Docker images) on every push.

Deploying: one Render web service from the included Blueprint (`render.yaml`) — build command `npm run build:render` , start command `npm run start:all` , health check `/api/health` , persistent disk at `/opt/data` for both databases, uploads and media. Full walkthrough: `docs/DEPLOY-RENDER.md` ; complete environment-variable reference: the repository README.

FAQ & troubleshooting

Do I have to connect my bank?

No — and the product never asks. Subscrr works on entries you create or documents you scan. That is a deliberate privacy stance, not a missing feature.

What happens if the AI misreads a receipt?

The result card is fully editable before saving, and a confidence meter flags low-certainty reads. The scanner never invents values: unknown amounts come back as zero with a note, non-subscription documents are rejected with a clear error.

Does pausing delete history?

No. Paused subscriptions are excluded from all totals but kept in the ledger; resuming restores everything instantly.

How do alerts reach me?

In-app notifications always; email whenever a provider is configured (Resend or any SMTP — Gmail app passwords work). Each alert type can be toggled per channel in Settings.

I didn't get a reset email.

Check spam first. If the deployment has no email provider configured, the platform logs reset links instead of sending them — the operator needs to add `SMTP_*` or `RESEND_API_KEY` environment variables. `/api/health` reports the active email transport.

"Too many requests" on login?

The brute-force limiter did its job. Wait for the window to reset or follow the on-screen cool-off message. Page-load session checks are budgeted separately and never cause this.

Can I use it in two currencies?

Yes — that's the point. Each subscription keeps its original currency; your dashboard converts everything to your display currency with live rates.

Where does my data live, and how do I leave?

In SQLite files on the deployment's persistent disk (self-hostable by design). Export the full ledger to CSV/JSON anytime from Subscriptions or Settings — the file is complete and standard.

Who can see my subscriptions?

Only you. Even platform admins see counts, not contents — the admin portal deliberately exposes no per-user subscription data.

What's on the roadmap?

Bank auto-sync (opt-in aggregation for those who want it), per-file signed URLs for receipts, and multi-instance scaling via Postgres + object storage — the current single-file design is the deliberate v1 trade-off for privacy and simplicity.



Subscrr

Know what you pay. Before you pay it.

Quick reference

App sign-in: `/login` · CMS admin: `/admin` · API health: `/api/health`

Data export: Subscriptions → Export CSV/JSON · Alerts: in-app + email, per-type toggles

v2.0 · September 2026 · Subscrr — Complete Product & Knowledge Guide · 19 pages · This document was generated from the live product; every screenshot is real.